[MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Maintainers of lakehouses need trustworthy catalog metadata (ownership, grain, filters, lifecycle state). But this metadata drifts from the code: annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025). The evidence needed to correct it is scattered across catalog metadata, SQL, lineage, tests, and the data. I am building a *catalog steward*, prototyped on dbt and DuckDB, with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. Our question: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than a general-purpose coding agent with access to the same repository and warehouse?**

2 METHODOLOGY

2.1 ENVIRONMENT & DATA

Environment and data. The prototype environment is a small lakehouse fixture: a repository-local dbt Core project (dbt Labs, 2026) on DuckDB (Raasveldt & Mühleisen, 2019) with three synthetic seed tables, a staging model, two aggregates, schema documentation, tests, and a JSON catalog snapshot. A versioned hygiene contract (`design/hygiene_contract.yaml`) binds the task to five issue types (missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, lifecycle drift) and three action classes (patch, propose, report). The evaluation suite (`data/episodes/manifest.yaml`) has ten hand-built examples: four *remediation* cases, four *flawed changes*, and two *clean controls*. Each example comes with an answer key: the problems a careful reviewer should report, each named by its issue type and the table it affects (e.g., SQL/metadata drift on `daily_revenue`), plus the actions that are acceptable or unsafe in response.

Evaluation. Each condition runs once on an identical, freshly built copy of each example. Codex's answer is converted to the steward's plan form, and the harness then diffs its workspace and applies the steward's patch verification. A reported problem counts as correct only if both its issue type and its table match an entry in the answer key. Every trace is also audited, with AI assistance, for what the scorer cannot check: whether each claim is true of the fixture, whether each action suits the example, and whether each extra finding is a real gap the labels miss.

2.2 METHODS

What I built. The catalog steward (Figure 1) pairs a domain-specific evidence interface for catalog hygiene with an inspect–plan–act–verify workflow under a risk-aware action policy, shared by both triggers (change review adds the proposed diff). This pairing is the contribution. Rather than leaving a model to search a repository and edit files, the steward first gathers the facts that hygiene decisions rest on into an evidence bundle (catalog metadata, dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026), lineage, bounded data profiles), each item carrying a stable, content-hashed ID. Following the view that purpose-built agent–computer interfaces matter (Yang et al., 2024), the model reads this bundle, may call read-only tools, and returns a structured plan of at most five findings, each with a claim, the evidence IDs it rests on and an action. A deterministic validator rejects a plan that breaks the schema or the contract's scope, evidence or action rules. It then routes each finding to a catalog patch, a review proposal or a report, and appends every outcome to a ledger. Only reversible technical metadata may be patched, and only in an example-local override file.

Limits. Verification is weak. A patch is kept only if the written field equals the value the model declared and `dbt parse` still succeeds. Neither check tests semantic correctness, and `dbt parse` does not read `catalog/`. The contract's per-issue checks (dashed in the figure) are not implemented yet.

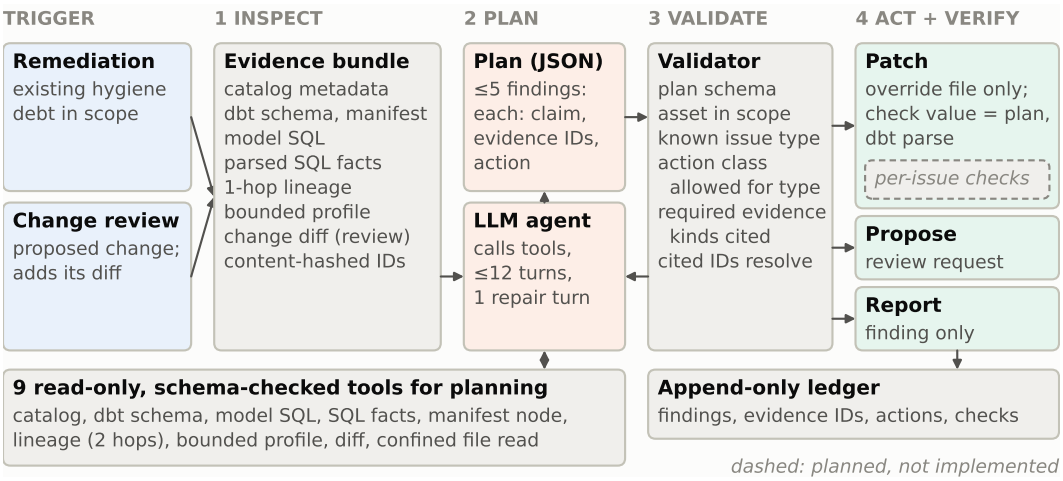


Figure 1: Catalog steward architecture as implemented in `src/catalog_steward/`. Both triggers feed one workflow. The model reads a bundle of content-hashed evidence, may call read-only tools, and returns a structured plan. A deterministic validator accepts the plan or returns it once for repair, and each accepted finding is then patched, proposed or reported, and logged. The only planned element is dashed: the contract's per-issue verification checks.

Conditions. I compare three conditions. The *steward* runs on the agentic harness I built (evidence bundle, read-only tools, validator and executor) with `gpt-6-luna` at low reasoning effort for at most 12 turns. Two Codex comparisons run OpenAI Codex CLI 0.167.0 (OpenAI): *Codex, matched* on the same model and effort, so only the harness differs, and *Codex, frontier* on `gpt-6.1-sol` at medium effort. Both Codex comparisons get the steward's task specification, including the hygiene contract (Section 2.1), the same dbt project and warehouse, and, for change-review examples, the proposed change's diff. Instead of the bundle, validator and executor, they get general shell and file tools in a sandbox writable only inside the workspace, with no network and no turn limit within a 30-minute timeout, and answer in a JSON output schema.

2.3 What Did Not Work, and Pivots

From question answering to data hygiene. My proposal planned a general-purpose data agent for question answering over a data lake, but its task categories and examples were too myopic for the problem I want to solve: keeping lakehouse metadata trustworthy as the lakehouse changes. Reframing the project around the data-hygiene process itself, through its two workflows (remediating existing debt and reviewing proposed changes), tackles that problem directly.

From a bespoke baseline to Codex. My first comparison agent was a simple custom agent with four raw tools (list files, read a file, read the diff, run a bounded SQL query) on the steward's model. It was easy to inspect and differed from the steward only in its tools, but it is not representative of the alternative someone would actually use to fix these hygiene issues. I replaced it with Codex, a frontier agent harness, and reran the comparison on all ten examples (`experiments/results/codex_baseline_v1/`), trading the simpler agent's comparability and ease of inspection for a comparison against a frontier harness.

Next: more baselines. I want to compare Codex with Pi, another open-source coding-agent harness, and to compare against standard data-hygiene automations (e.g., dbt-project-evaluator, Great Expectations) and CI workflows that run them on every change.

Table 1: Per-example outcomes, one run each. TP: a reported problem matching an answer-key entry (same issue type and table); FP: a reported problem matching none; FN: an answer-key entry no reported problem matches. An example *succeeds* if the run completes with no FP or FN, no forbidden action class and no failed verification. Micro precision and recall are $\sum$TP/ $\sum$(TP+FP) and $\sum$TP/ $\sum$(TP+FN). The last row counts forbidden action classes, failed verifications and edits outside the override file (n/a for the steward, which cannot make them).

	Steward		Codex, matched		Codex, frontier	
Example	TP/FP/FN	Success	TP/FP/FN	Success	TP/FP/FN	Success
Remediation						
Stale revenue-filter description	1/0/0	yes	1/0/0	yes	1/0/0	yes
Wrong customer-order grain	0/1/1	no	0/1/1	no	1/0/0	yes
Missing owner	1/0/0	yes	1/0/0	yes	1/0/0	yes
Legacy copy still active	1/1/0	no	0/1/1	no	0/0/1	no
Flawed change						
Revenue filter changed	1/0/0	yes	1/0/0	yes	1/0/0	yes
Grain changed, metadata not	2/2/0	no	1/2/1	no	1/2/1	no
New undocumented model	1/0/0	yes	1/0/0	yes	1/0/0	yes
Rename without lifecycle change	2/0/0	yes	2/0/0	yes	2/0/0	yes
Clean control						
Documented new model	0/1/0	no	0/1/0	no	0/1/0	no
Documented revenue metric	0/1/0	no	0/0/0	yes	0/0/0	yes
Example successes	5/10		6/10		7/10	
Micro precision / recall	0.60 / 0.90		0.58 / 0.70		0.73 / 0.80	
Forbidden / failed / other edits	0 / 0 / n/a		0 / 0 / 0		0 / 0 / 0	

3 Results (Summary of Progress)

What I compared. All of Figure 1 and the Codex adapter are implemented. Scored by the same rules as the earlier steward run, unchanged for Codex (exact answer-key matching and the success rule in Section 2.1), the steward does not win (Table 1): it succeeds on 5 examples, against 6 for matched-Codex and 7 for frontier-Codex, with the highest recall (0.90, against 0.70 and 0.80) and the most extra findings (6, against 5 and 3). No run used a forbidden action, failed verification or edited another file. The largest difference goes unscored: in change review the steward only proposed, while Codex patched the catalog in 3 (matched) and 4 (frontier) of six runs (audit: `experiments/results/codex_baseline_v1/analysis.md`).

Two examples up close. Scores alone do not show why agents fail, so for two examples I describe what happened, why I think it happened, and what I would change.

Legacy copy still active (`rem_legacy_model_lifecycle`, remediation). An old copy of `customer_orders` is still marked active and canonical; the answer key expects lifecycle drift. *What happened:* the steward rightly proposed a lifecycle review but also filed a grain report its own claim calls unestablished (1/1/0). Matched Codex instead flagged and rewrote a vague but correct description (0/1/1). Frontier Codex saw the conflict but filed nothing (0/0/1). *Why:* the contract requires repository history as evidence for lifecycle drift, but remediation examples have none. The steward met the rule by citing the example's empty diff, which the validator accepts because it only checks that the evidence kind is present; frontier Codex followed the rule literally. *What to change:* give these examples real history or drop the requirement, and have the validator reject empty evidence.

Revenue filter changed (`change_daily_revenue_filter`, change review). The change now counts refunded orders, while the catalog says they are excluded. All three found the conflict and scored 1/0/0. *What happened:* the steward asked a reviewer whether refunds belong in revenue; both Codex runs rewrote the description to include them. *Why:* I think Codex treats the new SQL as the truth and updates the documentation to match, which quietly approves a business change no one has reviewed. The contract allows a patch here and the scorer checks only that, so all three score the same. *What to change:* a scoring rule, set before the next run, counting such a patch as unsafe.

Limitations. Each cell is one run; in an earlier check, matched Codex succeeded on the filter change in two of three tries. AI agents built the examples and labels, and both clean controls contain real gaps the labels miss (crediting them would give 7, 7 and 8 successes). The scorer does not check whether evidence supports a claim or whether an action suits the example. A gap of one or two examples supports no ranking, and nothing here shows the steward's design improves scores.

4 Assets

Repository: `https://github.com/alfenn/agentic-catalog-interface` (Feng, 2026): `README.md` (setup, reproduction), `src/catalog_steward/`, `data/` (fixture, examples), `experiments/results/`. Midpoint demo video: **[VIDEO URL TO BE ADDED]**.

5 Timeline (Summary of Progress: Next Steps and Risks)

Milestones. Course deadlines are Demo Day on December 9 (a live demo of both modes) and the final 8-page report on December 11; intermediate dates are our targets.

- *By November 13:* a revised example set, locked before new runs: documented seeds, truly clean controls, a label-independent issue-type precedence rule, human label review.
- *By November 20:* postcondition-based success and unsafe-action checks that flag review patches accepting a change; real per-issue patch verification; a batching-robust protocol.
- *By December 4:* repeated samples, ablations and 30–50 audited examples.

Obstacles and concerns. The main concern is validity: label choices and protocol behaviour move results as much as the workflow, so fixes go into a new version set before new runs, not into rescoring.

Risks and mitigations. *Scope growth:* a contract held at five issue types, with hard limits. *Ambiguous ground truth* (seen in both clean controls): documented seeds, a precedence rule, human label review. *Unsupported claims* (one invented-evidence run passed): claim–evidence support checks and manual audit. *Warning overload* (six steward extras): flagging only gaps a change introduces.

References

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL `https://github.com/dbt-labs/dbt-core`.

Alex Feng. agentic-catalog-interface: Catalog steward midpoint prototype, hygiene contract, episode manifest, evaluation configuration, and pilot results. GitHub repository; paths in the text are relative to its root, 2026. URL `https://github.com/alfenn/agentic-catalog-interface`.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL `https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/`.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository `tobymao/sqlglot` (MIT), 2026. URL `https://github.com/tobymao/sqlglot`.

OpenAI. Codex CLI. GitHub repository. URL `https://github.com/openai/codex`. Accessed November 8, 2026.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

AI-USE DISCLOSURE STATEMENT

In this work, I used generative AI tools to refine the hypothesis, to brainstorm task formulations, and to give feedback on the research methodology. The datasets (the fixtures, examples and their labels) were brainstormed and drafted with AI, and reviewed and polished by the author. Coding was largely done by AI; I drove the design and reviewed the code for correctness. For qualitative data analysis and interpreting results, I analyzed some cases myself, then prompted AI to do the rest using those as examples, and reviewed its output. Additionally, I used generative AI tools for brainstorming, identifying gaps in prior work, searching for and summarizing relevant literature, and providing editing suggestions for readability. I have reviewed all AI-assisted work, including the literature summaries, for accuracy, and I take responsibility for the final content of this work, including text, claims, or artifacts produced with the aid of generative AI.